

0.104. Une lettre qui tire — un petit jeu : les W tombent

Tout ensemble, dans un petit jeu : le A, en bas, va de côté et tire vers le haut ; trois W tombent. Un W touché : un point, et il repart d'en haut. Un W arrivé en bas : une vie de moins. PERDU, et START.

1. Ce qu'il faut comprendre

Un petit jeu complet, avec tout ce qu'on a vu. Le A est en bas, sur la ligne 15, au-dessus d'un sol de « # ». Il va à gauche et à droite, et tire **vers le haut**. Trois **W** tombent du ciel, une case toutes les 600 ms. Un W touché : un point, et il repart d'en haut. Un W qui arrive en bas : une vie de moins.

Ce qui vient d'avant : le tir qui avance et regarde sa case d'arrivée (le 0.101), la cible touchée (le 0.102), trois tirs dans un tableau de `struct` avec `chaque(250)` (le 0.102.1), les vies, PERDU et START (le 0.103), les ennemis dans un tableau (le 0.85).

Plus simple qu'avant, sur deux points. Le A ne va que de côté : sa ligne est toujours 15, pas besoin de `ay`. Et les tirs vont **toujours vers le haut** : le `struct Tir` perd son sens (`dx`, `dy`), l'arrivée d'un tir est simplement la ligne `y - 1`.

Ce qui est nouveau ici (1) : plusieurs cibles. Au 0.102, une seule cible : `nx == bx && ny == by`. Ici, trois W : une boucle sur `j` compare l'arrivée du tir à **chaque** W. Si l'un est là : il disparaît, `placerW(j)` le renvoie en haut, et `touche = 1`. Après la boucle, `touche` dit si le tir a touché quelque chose.

Ce qui est nouveau ici (2) : un W ne tombe pas sur un tir. Si la case sous un W n'est pas vide (un tir, ou un autre W), le W **attend** : il ne descend pas à ce tour-ci. Au pas suivant du tir, c'est le tir qui monte sur lui : touché. Sans cette attente, le W écraserait le dessin du tir, et le tir, en avançant, effacerait... le W.

Arrivé en bas : quand l'arrivée d'un W est la ligne 15, celle du A, il ne s'y pose pas : il disparaît, repart d'en haut, et le joueur perd une vie.

`nouvellePartie()` : tout ce qu'il faut pour commencer (vider le terrain, arrêter les tirs, placer les W et le A, 3 vies, score 0) est rangé dans une fonction. On l'appelle au début **et** quand on appuie sur START : écrit une seule fois, fait deux fois.

Essaie : fais tomber les W plus vite (`chaque(400)`), ajoute un quatrième W (`Ennemi ennemis[4]`, et `j < 4` partout), ou fais accélérer la chute quand le score monte.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <hasard>` → [0.81.2. La fonction hasard\(\) — un nombre au hasard](#)
- `#include <lire>` → [0.68.4. La fonction lire\(\) — lire une case](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <effacer>` → [0.16.3. La fonction effacer\(\) — effacer des cases](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <chaque>` → [0.29.3. La fonction chaque\(\) — toutes les n millisecondes](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <note>` → [0.82.2. La fonction note\(\) — jouer une note](#)
- `#include <bruit>` → [0.83.1. La fonction bruit\(\) — un bruit](#)
- `#include <reste>` → [0.8.4. #include <reste> — a % b](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **2355 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	96 o	← la console, à l'allumage — la mise en route de la console
chaque_minuteur()	124 o	← main() — une fonction fournie par le langage
placerW()	83 o	← nouvellePartie(), main() — ta fonction
nouvellePartie()	127 o	← main() — ta fonction
main()	1067 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EffacerCases, EcrireTexte, EcrireNombre — attend de pouvoir écrire une case
AdresseCase	10 o	← lire(), poser(), effacer(), texte() — trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← effacer() — efface des cases
EcrireTexte	10 o	← texte() — écrit un texte, lettre par lettre
EcrireNombre	33 o	← nombre() — écrit un nombre en chiffres
Reste	10 o	← placerW() — le reste d'une division
CopierTuiles	103 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
Hasard	25 o	← hasard() — tire un nombre au hasard
LireManette	37 o	← bouton() — lit les huit boutons
Tuiles	384 o	← CopierTuiles — 24 lettres sur 43
Notes	120 o	← note() — la table des notes de musique
"#####..."	20 o	← texte() — un texte gravé
"SCORE"	5 o	← texte() — un texte gravé
"VIES"	4 o	← texte() — un texte gravé
"."	1 o	← texte() — un texte gravé
"."	1 o	← texte() — un texte gravé
"PERDU"	5 o	← texte() — un texte gravé

Lettres gravées : ACDEIOPRSUVW0123456789.#

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, Diviser, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, EffacerFond, EffacerPanneau, DonneesTransfert

Fonctions du langage appelées :

effacer()	x 6	157 o sur place, dans nouvellePartie(), main() + AdresseCase (10 o), EffacerCases (12 o)
texte()	x 6	80 o sur place, dans main() + AdresseCase (10 o), EcrireTexte (10 o)
poser()	x 4	60 o sur place, dans placerW(), nouvellePartie(), main() + AdresseCase (10 o)
note()	x 1	56 o sur place, dans main() + Notes (120 o)

```

lire()           x 4    52 o sur place, dans placerW(), main() + AdresseCase (10 o)
nombre()        x 4    40 o sur place, dans nouvellePartie(), main() + EcrireNombre (33 o)
bruit()         x 1    39 o sur place, dans main()
bouton()        x 4    20 o sur place, dans main() + LireManette (37 o)
hasard()        x 1     3 o sur place, dans placerW() + Hasard (25 o)
image()         x 1     3 o sur place, dans main() + AttendreImage (28 o)
images()        x 1     2 o sur place, dans chaque_minuteur()

Tes fonctions appelées :
chaque_minuteur() x 4    43 o sur place, dans main()
placerW()         x 3    21 o sur place, dans nouvellePartie(), main()
nouvellePartie() x 2     6 o sur place, dans main()

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// ---- UN PETIT JEU DE TIR : les W tombent ----
// Tout ce qu'on a vu, dans un vrai petit jeu. Le A, en bas, va à gauche et
// à droite, et tire vers le HAUT. Trois W tombent du ciel. Un W touché :
// un point, et il repart d'en haut. Un W qui arrive en bas : une vie de moins.
// ALPHABET[0] : le A   ALPHABET[22] : le W
//
// ligne 0 :   W       W       W       les W partent d'ici, et tombent
// ...      :   |               un tir du A monte
// ligne 15 :   A               le A : il ne va que de côté
// ligne 16 : #####           le sol
// ligne 17 : SCORE 0   VIES 3

// ---- un tir : il monte toujours, il n'a donc plus besoin de sens ----

#include <hasard>      // tire un nombre au hasard
#include <lire>        // lit la tuile posée sur une case
#include <poser>       // pose une tuile sur une case du fond
#include <ALPHABET>    // les lettres de la police : ALPHABET[0] est le A
#include <effacer>     // efface des cases, ou tout le fond
#include <nombre>      // écrit un nombre en chiffres
#include <texte>       // écrit un texte à l'écran
#include <chaque>      // répond 1 toutes les n millisecondes
#include <bouton>      // lit un bouton de la manette
#include <note>       // joue une note
#include <bruit>       // joue un bruit
#include <reste>      // a % b, sauf par 1, 2, 4, 8, 16... écrits en clair

```

2. UN ENREGISTREMENT — TIR

```

struct Tir {
    uint8_t x, y;           // sa place
    uint8_t vole;          // 1 = en vol ; 0 = libre
};
// ---- un ennemi W : sa place ----

```

3. UN ENREGISTREMENT — ENNEMI

```

struct Ennemi {
    uint8_t x, y;
};

```

4. UN TABLEAU — TIRS

```

Tir tirs[3];           // trois tirs à la fois (le 0.102.1)

```

5. UN TABLEAU — ENNEMIS

```

Ennemi ennemis[3];     // trois W (le 0.85)

```

6. UNE VARIABLE — AX

```
uint8_t ax = 9;           // le A : sa colonne. Sa ligne est toujours 15.
```

7. UNE VARIABLE — NX

```
uint8_t nx = 0;           // une case d'ARRIVÉE, calculée avant de bouger
```

8. UNE VARIABLE — NY

```
uint8_t ny = 0;
```

9. UNE VARIABLE — SCORE

```
uint8_t score = 0;
```

10. UNE VARIABLE — VIES

```
uint8_t vies = 3;
```

11. UNE VARIABLE — ETAT

```
uint8_t etat = 0;         // 0 = on joue, 1 = perdu (le 0.84)
```

12. UNE VARIABLE — LANCE

```
uint8_t lance = 0;        // 1 = un tir vient de partir, à ce tour-ci
```

13. UNE VARIABLE — TOUCHE

```
uint8_t touche = 0;       // 1 = le tir qu'on regarde vient de toucher un W
```

```
// ---- le W numéro j repart d'en haut : ligne 0, colonne au hasard ----
```

14. UNE FONCTION — PLACERW()

```
void placerW(uint8_t j) {  
    do {  
        ennemis[j].x = hasard() % 20;    // colonne 0 à 19...  
        ennemis[j].y = 0;                // ...sur la ligne 0...  
    } while (lire(ennemis[j].x, 0) != 0); // ...sur une case vide  
    poser(ennemis[j].x, 0, ALPHABET[22]);  
}  
  
// ---- une nouvelle partie : le terrain vidé, tout à sa place ----
```

15. UNE FONCTION — NOUVELLEPARTIE()

```
void nouvellePartie() {
    for (uint8_t l = 0; l < 16; l++) {    // les lignes 0 à 15 : le terrain
        effacer(0, l, 20);
    }
    for (uint8_t i = 0; i < 3; i++) {
        tirs[i].vole = 0;                // aucun tir en vol...
        placerW(i);                      // ...et les trois W en haut
    }
    ax = 9;
    poser(ax, 15, ALPHABET[0]);          // le A, en bas au milieu
    score = 0;
    vies = 3;
    nombre(6, 17, score);
    nombre(15, 17, vies);
    etat = 0;
}
```

16. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    texte(0, 16, "#####"); // le sol : 20 « # »
    texte(0, 17, "SCORE");
    texte(10, 17, "VIES");
    nouvellePartie();

    while (true) {
        image();

        if (etat == 0) { // ===== ON JOUE

            // ---- le A va à gauche et à droite, sur la ligne 15 ----
            if (chaque(120)) {
                nx = ax;
                if (bouton(DROITE)) {
                    nx = ax + 1;
                } else if (bouton(GAUCHE)) {
                    nx = ax - 1; // en colonne 0 : 255, refusé juste après
                }
                if (nx < 20 && nx != ax) {
                    effacer(ax, 15, 1);
                    ax = nx;
                    poser(ax, 15, ALPHABET[0]);
                }
            }

            // ---- A tire vers le haut, au plus toutes les 250 ms (le 0.102.1) ----
            if (chaque(250)) {
                if (bouton(A) && lire(ax, 14) == 0) { // la case au-dessus du A est vide
                    lance = 0;
                    for (uint8_t i = 0; i < 3; i++) {
                        if (lance == 0 && tirs[i].vole == 0) {
                            tirs[i].vole = 1;
                            tirs[i].x = ax;
                            tirs[i].y = 14;
                            texte(ax, 14, ".");
                            lance = 1;
                        }
                    }
                }
            }
        }

        // ---- chaque tir monte d'une case ----
        if (chaque(80)) {
            for (uint8_t i = 0; i < 3; i++) {
                if (tirs[i].vole == 1) {
                    nx = tirs[i].x;
                    ny = tirs[i].y - 1; // monter, c'est retirer 1 à la ligne
                    effacer(tirs[i].x, tirs[i].y, 1);
                    // Un W sur l'arrivée ? On les regarde tous les trois.
                    touche = 0;
                    for (uint8_t j = 0; j < 3; j++) {
                        if (ny < 17 && ennemis[j].x == nx && ennemis[j].y == ny) {
                            effacer(nx, ny, 1); // le W disparaît...
                            placerW(j); // ...et repart d'en haut
                            touche = 1;
                        }
                    }
                }
            }
            if (touche == 1) {
                tirs[i].vole = 0; // 1. un W : touché !
                score = score + 1;
                nombre(6, 17, score);
                note(1, D05, 10, 12);
            } else if (ny >= 17) {
```

```

        tirs[i].vole = 0;          // 2. sorti par le haut (0 - 1 = 255)
    } else if (lire(nx, ny) != 0) {
        tirs[i].vole = 0;          // 3. autre chose : il s'arrête
    } else {
        tirs[i].y = ny;            // 4. une case vide : il y monte
        texte(nx, ny, ".");
    }
}
}
}

// ---- les W descendent d'une case, toutes les 600 ms ----
if (chaque(600)) {
    for (uint8_t j = 0; j < 3; j++) {
        nx = ennemis[j].x;
        ny = ennemis[j].y + 1;
        if (ny == 15) {
            // Arrivé en bas, sur la ligne du A : une vie de moins.
            effacer(nx, ennemis[j].y, 1);
            placerW(j);
            vies = vies - 1;
            nombre(15, 17, vies);
            bruit(8, 12);
            if (vies == 0) {
                etat = 1;
                texte(7, 8, "PERDU");
            }
        } else if (lire(nx, ny) == 0) {
            effacer(nx, ennemis[j].y, 1); // la case en dessous est vide : il y va
            ennemis[j].y = ny;
            poser(nx, ny, ALPHABET[22]);
        }
        // Sinon, un tir (ou un autre W) est juste dessous : le W attend.
        // Au prochain pas du tir, c'est le tir qui monte sur lui : touché.
    }
}

} else {
    // ===== PERDU
    if (bouton(START)) {
        nouvellePartie(); // tout recommence
    }
}
}
}
}

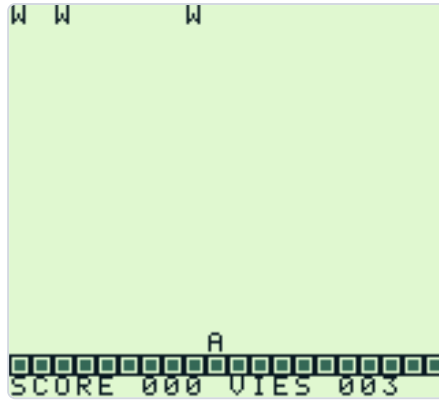
```

3. Ce qui se passe, étape par étape

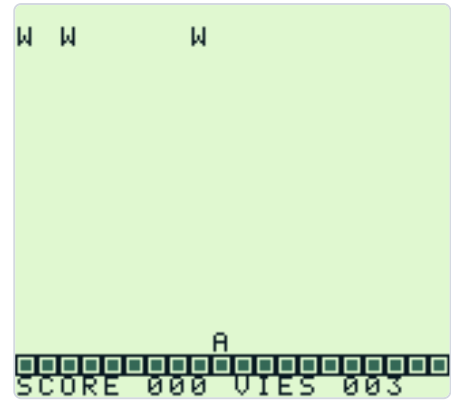
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



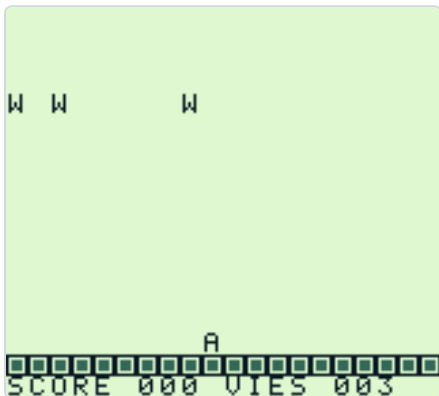
la toute première image — la console vient de démarrer — image 1



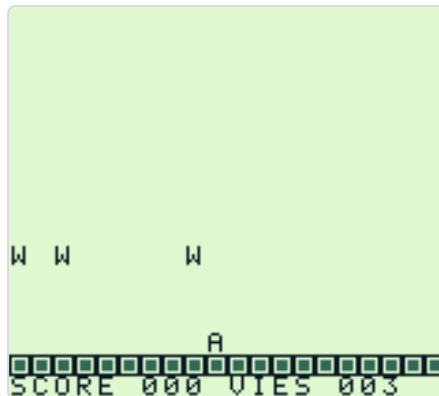
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



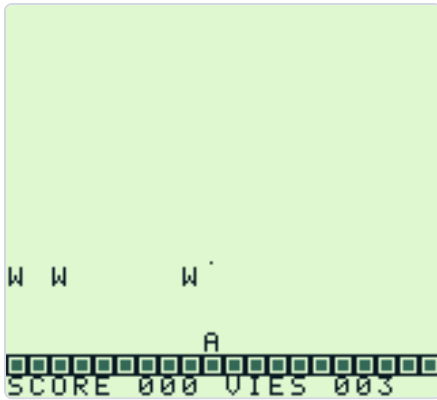
au bout de trois secondes — image 180



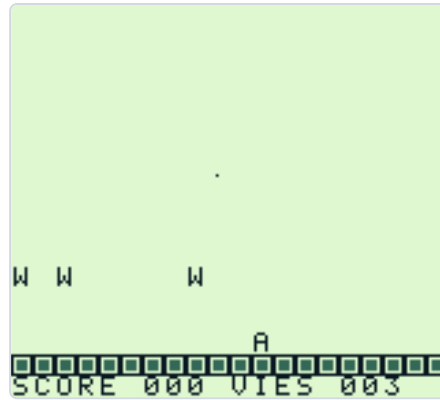
au bout de sept secondes — image 420

4. Ce que les touches changent

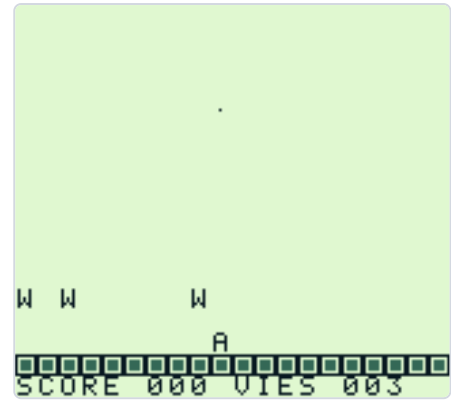
Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



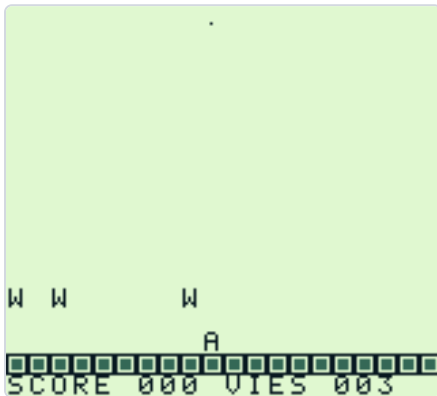
A — l'écran a changé



DROITE — l'écran a changé



GAUCHE — l'écran a changé



START — l'écran a changé

5. Ce qu'on doit voir

Trois W tombent. Le A, en bas, va de côté et tire vers le haut : un W touché rapporte un point et repart d'en haut ; un W arrivé en bas coûte une vie. À 0 : PERDU ; START relance.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ au départ : trois W en haut, le A en (9, 15), le sol, SCORE 000
- ✓ en visant, le A touche des W : le score monte (score 5)
- ✓ un W touché repart d'en haut : toujours trois W
- ✓ sans tirer, les W arrivent en bas : les vies tombent à 0, PERDU
- ✓ START : une nouvelle partie, 3 vies, score 0

Ensuite : **0.105. Une lettre qui tire — le tir casse le X** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++